PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 03 -

MTTR BAIXO COM PAGERDUTY

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....5
MERCADO, CASES E TENDÊNCIAS 19
REFERÊNCIAS.....21

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AÍ?

Nas aulas anteriores, estabelecemos as fundações para um gerenciamento de incidentes robusto. Nesta aula, aprendemos o que é um incidente e como diferentes frameworks o definem. Exploramos como detectar incidentes de forma proativa através de monitoramento inteligente e combater a fadiga de alertas. Agora, chegamos a uma questão crítica: quando um incidente inevitavelmente ocorre, como garantir que ele seja resolvido o mais rapidamente possível?

A resposta está em um conceito fundamental para qualquer operação de TI: o MTTR (Mean Time To Resolution), ou Tempo Médio para Resolução. O MTTR não é apenas uma métrica; é um indicador direto do impacto financeiro, da satisfação do cliente e da saúde operacional da sua organização. Uma redução de uma hora no MTTR pode economizar milhões de dólares em receita perdida, especialmente para serviços críticos. Esta aula explora não apenas o conceito de MTTR, mas como mensurá-lo rigorosamente, implementá-lo em sua organização e mais importante, como reduzi-lo através de estratégias comprovadas e ferramentas modernas como o PagerDuty.

O PagerDuty é muito mais que um sistema de alertas; é uma plataforma de gerenciamento de incidentes que foi projetada desde o início para reduzir o MTTR através de automação, orquestração e inteligência. Vamos explorar como suas funcionalidades se alinham com as melhores práticas de redução de MTTR e como implementá-las em sua operação.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

Esta aula focará em reduzir o MTTR com PagerDuty: definir a métrica, decompor em detecção/resposta/diagnóstico/remediação/verificação, ensinar medição e estratégias avançadas como automação de runbooks e observabilidade, preparando para operações TI eficientes e de baixo impacto financeiro.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Entendendo o MTTR: Definição, Contexto e Importância

O MTTR (Mean Time To Resolution) é uma métrica que mede o tempo médio decorrido desde o momento em que um incidente é detectado até o momento em que ele é totalmente resolvido e o serviço retorna ao seu estado normal de operação. É importante notar que o MTTR começa quando o incidente é detectado, não quando ele realmente começou. Um incidente pode estar ocorrendo há horas sem ser detectado; o MTTR conta apenas a partir do momento em que alguém, ou um sistema automatizado, percebe que algo está errado.

A importância do MTTR vai muito além de ser um número em um relatório. Considere uma plataforma de e-commerce que processa US$ 10 mil em vendas por minuto. Um incidente que causa uma hora de downtime resulta em US$ 600 mil em receita perdida, sem contar a perda de confiança do cliente, possíveis multas por violação de SLA e danos à reputação da marca. Reduzir o MTTR de 60 minutos para 30 minutos não apenas reduz a perda para US$ 300 mil, mas também demonstra competência operacional e confiabilidade.

O MTTR também é um reflexo da maturidade operacional da organização. Equipes com MTTR baixo (menos de 15 minutos) geralmente têm: processos bem definidos e documentados, automação significativa em resposta a incidentes, equipes bem treinadas e com conhecimento profundo dos sistemas, ferramentas de observabilidade e diagnóstico sofisticadas, além de cultura de aprendizado contínuo através de postmortems.

Medição do MTTR: Metodologia e Desafios

Embora o conceito de MTTR pareça simples, sua medição rigorosa apresenta desafios significativos que muitas organizações subestimam.

Componentes do MTTR

Para entender completamente o MTTR, é útil decompô-lo em seus componentes:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Tempo de Detecção (Detection Time): o tempo desde que o incidente começou até que foi detectado. Este é frequentemente o componente mais longo e o mais negligenciado. Um incidente pode estar afetando usuários por 30 minutos antes que o sistema de monitoramento o detecte.

Tempo de Resposta (Response Time): o tempo desde a detecção até que alguém (o plantonista) reconheça o alerta e comece a investigar. Em uma operação bem organizada, isso deve ser medido em minutos. Em operações caóticas, pode ser horas.

Tempo de Diagnóstico (Diagnosis Time): o tempo gasto investigando o problema, coletando dados, analisando logs e traces, e identificando a causa raiz. Este é frequentemente o componente mais longo e onde a observabilidade e as ferramentas de diagnóstico têm maior impacto.

Tempo de Remediação (Remediation Time): o tempo gasto implementando a solução. Isso pode variar enormemente dependendo da natureza do problema: desde alguns segundos para reiniciar um serviço, até horas por causa de um rollback de um deployment problemático.

Tempo de Verificação (Verification Time): o tempo gasto confirmando que a solução funcionou e que o serviço retornou ao normal. Isso inclui testes, monitoramento de métricas e confirmação de que os usuários não estão mais afetados.

Fórmula do MTTR Total:

MTTR Total = Tempo de Detecção + Tempo de Resposta + Tempo de Diagnóstico + Tempo de Remediação + Tempo de Verificação.

Metodologia de Medição

A medição rigorosa do MTTR exige definições claras e coleta de dados consistente. Aqui está uma metodologia comprovada:

Passo 1: Definir o Início e o Fim

- **Início**: o momento em que o alerta é gerado ou o incidente é detectado pelo sistema de monitoramento. Algumas organizações usam o timestamp do primeiro alerta relacionado ao incidente.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Fim**: o momento em que o serviço retorna ao seu SLO, e é confirmado que não há mais impacto no usuário. Isso é diferente de “o problema foi corrigido no código”; é quando o usuário não está mais afetado.

Passo 2: Registrar Eventos Importantes

Para cada incidente, registre os seguintes timestamps:

- Tempo de detecção: quando o alerta foi gerado.
- Tempo de reconhecimento: quando o plantonista reconheceu o alerta.
- Tempo de início da investigação.
- Tempo de identificação da causa raiz.
- Tempo de início da remediação.
- Tempo de conclusão da remediação.
- Tempo de confirmação de resolução.

Passo 3: Categorizar Incidentes

Nem todos os incidentes são iguais. Um incidente que afeta 1% dos usuários por 5 minutos não deveria ser comparado com um que afeta 100% dos usuários por 2 horas. Categorize incidentes por:

- Severidade: crítico (impacto total).
- Alto: impacto significativo.
- Médio.
- Baixo: falha de infraestrutura, bug de código, problema de configuração, etc.
- Serviço afetado: diferentes serviços podem ter diferentes MTTR esperados.

Passo 4: Calcular MTTR por Categoria

Calcule o MTTR médio para cada categoria. Por exemplo:

- MTTR para incidentes críticos: 15 minutos.
- MTTR para incidentes de infraestrutura: 20 minutos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- MTTR para incidentes de código: 45 minutos.

Isso fornece uma visão mais nuançada do desempenho operacional.

Desafios Comuns na Medição

A medição do MTTR enfrenta vários desafios práticos:

- Incidentes Não Detectados: se um incidente não é detectado pelo sistema de monitoramento, ele não é registrado, distorcendo o MTTR médio. Uma organização pode ter um MTTR de 20 minutos para incidentes detectados, mas estar perdendo US$ 1 milhão por hora em incidentes não detectados.
- Definição Ambígua de “Resolvido”: o que significa “resolvido”? O serviço retornou ao normal? Os usuários não estão mais afetados? O problema foi corrigido permanentemente? Diferentes equipes podem ter definições diferentes, levando a inconsistências.
- Incidentes Parcialmente Resolvidos: às vezes, um incidente é parcialmente mitigado. Ex: tráfego é redirecionado para outro datacenter, enquanto a causa raiz ainda está sendo investigada. Qual é o MTTR neste caso?
- Incidentes Recorrentes: se o mesmo incidente ocorre novamente dias depois, é um novo incidente ou uma continuação do anterior? Isso afeta como o MTTR é calculado.

Estratégias Avançadas para Reduzir o MTTR

Reduzir o MTTR exige uma abordagem multifacetada que aborda cada componente do tempo total. As estratégias a seguir são baseadas em práticas comprovadas de organizações com MTTR muito baixo (menos de 15 minutos para incidentes críticos).

Reduzir o Tempo de Detecção Através de Observabilidade Avançada

O tempo de detecção é frequentemente o componente mais longo e o mais negligenciado do MTTR. Um incidente pode estar afetando usuários por 30 minutos antes de ser detectado, dobrando efetivamente o MTTR.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Metodologia de Implementação:

A redução do tempo de detecção começa com a compreensão de que nem todos os problemas geram alertas de monitoramento. Um usuário pode estar recebendo erros enquanto as métricas de infraestrutura parecem normais. Por isso, é essencial implementar múltiplas camadas de detecção:

- Monitoramento de Sintomas (Symptom Monitoring): em vez de monitorar apenas métricas de infraestrutura (CPU, memória), monitore sintomas que os usuários experimentam. Isso inclui taxa de erro da aplicação, latência de transações de negócio críticas, e taxa de sucesso de operações específicas. Ex: taxa de sucesso de checkout em um e-commerce.
- Detecção de Anomalias em Tempo Real: use baselines dinâmicos para detectar desvios do comportamento normal. Isso é particularmente eficaz para problemas que não têm um limiar óbvio. Ex: um aumento gradual na latência que não ultrapassa um limiar fixo, mas que é anormal para aquele horário do dia.
- Monitoramento de Experiência do Usuário (Real User Monitoring – RUM): Implemente RUM para capturar a experiência real dos usuários. Isso pode detectar problemas que não aparecem em métricas de infraestrutura. Ex: um problema de JavaScript no navegador que causa falhas silenciosas.
- Testes Sintéticos Contínuos: execute testes automatizados que simulam transações críticas de usuários. Ex: login, compra e envio de mensagem a cada minuto. Se um teste falhar, um incidente é detectado imediatamente.

Exemplo Prático - E-commerce:

Considere um site de e-commerce. Os alertas tradicionais monitoram: CPU do servidor: 45% (normal). Memória: 60% (normal). Taxa de erro HTTP: 0.1% (normal).

Mas há um problema: o serviço de pagamento está lento, causando timeouts em 5% das tentativas de checkout. Os usuários estão vendo erros, mas as métricas de infraestrutura parecem normais.

Com observabilidade avançada: o RUM detecta que a taxa de sucesso de checkout caiu de 99% para 94% e o alerta é disparado em 2 minutos. Teste Sintético tenta fazer uma compra de teste a cada minuto e falha, um alerta disparado em 1

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

minuto. Monitoramento de Sintomas mostra que a latência do serviço de pagamento aumentou de 200ms para 2000ms, um alerta é disparado em 3 minutos.

Sem essas camadas, o problema poderia passar despercebido por 30+ minutos até que um cliente reclamasse ou a taxa de erro geral ultrapassasse um limiar.

Desafios e Considerações:

A implementação de observabilidade avançada é cara em termos de volume de dados. RUM e testes sintéticos contínuos geram muitos dados. A solução é ser seletivo: implemente RUM apenas para transações críticas de negócio e execute testes sintéticos apenas para endpoints críticos.

Reduzir o Tempo de Resposta Através de Automação e Escalação Inteligente

O tempo de resposta é o tempo entre a detecção e o reconhecimento do alerta pelo plantonista. Em uma operação bem organizada, isso deve ser medido em minutos, não em horas.

Metodologia de Implementação:

A redução do tempo de resposta começa com a automação de tarefas que não exigem intervenção humana:

- **Automação de Remediação Imediata**: para problemas conhecidos com soluções padronizadas, implemente automação que tenta resolver o problema antes de notificar um humano. Exemplos incluem: reiniciar um serviço que travou, escalar automaticamente um serviço que está recebendo tráfego acima do normal, limpar cache que ficou corrompido.
- **Escalação Inteligente**: se a automação falhar ou se o problema exigir intervenção humana, escale para o plantonista apropriado. Isso significa não notificar o plantonista de banco de dados quando o problema é com o serviço de frontend.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Notificação Multicanal**: não confie em um único canal de notificação. Se o plantonista não reconhecer o alerta em Slack em 2 minutos, envie um SMS. Se não reconhecer em 5 minutos, faça uma chamada telefônica.

Exemplo Prático - Escalação Inteligente:

Considere um incidente onde um serviço de cache fica indisponível:

1. T=0: alerta é gerado “Cache Redis Indisponível”.
2. T=1 segundo: sistema tenta automação e reinicia o serviço Redis.
3. T=5 segundos: se o Redis não voltar, a automação tenta failover para um Redis secundário.
4. T=10 segundos: se o failover falhar, o alerta é escalado para o plantonista de infraestrutura via Slack.
5. T=2 minutos: se o plantonista não reconhecer o alerta em Slack, um SMS é enviado.
6. T=5 minutos: se ainda não reconhecido, uma chamada telefônica é feita.

Neste cenário, o tempo de resposta é no máximo 5 minutos, e há uma boa chance de que o problema foi automaticamente resolvido nos primeiros 10 segundos.

Desafios e Considerações:

A automação de remediação é poderosa, mas perigosa se mal implementada. Uma automação que reinicia um serviço crítico sem validação pode causar mais dano do que bem. A solução é implementar automação em camadas: primeiro tente ações de baixo risco (reiniciar), depois ações de médio risco (failover) e apenas como último recurso ações de alto risco (rollback de deployment).

Reduzir o Tempo de Diagnóstico Através de Contexto Pré-Compilado

O tempo de diagnóstico é frequentemente o componente mais longo do MTTR. Um engenheiro pode passar 30 minutos navegando entre dashboards, logs e traces para identificar a causa raiz. Reduzir esse tempo exige fornecer contexto relevante imediatamente quando o alerta é disparado.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Metodologia de Implementação:

O contexto pré-compilado significa que, quando um alerta é disparado, as informações mais relevantes já estão agregadas e apresentadas ao engenheiro:

- **Runbooks Automatizados**: um runbook é um guia passo-a-passo para diagnosticar e resolver um problema. Quando um alerta é disparado, o runbook correspondente é automaticamente apresentado ao engenheiro, junto com dados relevantes já coletados.
- **Histórico de Incidentes Similares**: se este incidente é similar a um incidente anterior, o sistema apresenta o histórico do incidente anterior, incluindo a causa raiz e a solução.
- **Dados de Contexto Enriquecidos**: o alerta não é apenas “Serviço X Indisponível”; é “Serviço X Indisponível afetando 15% dos usuários, começou após o deployment Y e 3 instâncias de 10 estão respondendo”.
- **Correlação Automática de Eventos:** o sistema correlaciona automaticamente eventos de múltiplas fontes, como logs, métricas, traces e eventos de deployment para identificar a causa raiz mais provável.

Exemplo Prático - Diagnóstico Acelerado:

Considere um incidente onde a latência da API aumenta. Sem contexto pré-compilado:

1. Engenheiro recebe alerta: “Latência da API P99 > 1000ms”.
2. Engenheiro abre Grafana para ver gráficos de latência.
3. Engenheiro abre Datadog para ver traces distribuídos.
4. Engenheiro abre o repositório de logs para ver erros.
5. Engenheiro verifica o histórico de deployments.
6. Engenheiro correlaciona manualmente: “Ah, a latência aumentou logo após o deployment Y. Deve ser um problema de código.”
7. Engenheiro revisa o código do deployment Y.
8. Engenheiro identifica a causa raiz: uma query de banco de dados N+1.

Tempo total: 35 minutos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Com contexto pré-compilado:

Engenheiro recebe alerta com contexto: “Latência da API P99 > 1000ms. Aumento começou após deployment Y (commit Z). Serviço de Usuários está vendo latência alta ao chamar Banco de Dados. Histórico: incidente similar em 15/11/2025 foi causado por query N+1 no serviço de Usuários.”

Engenheiro abre o runbook para “Latência Alta no Serviço de Usuários”.

Runbook apresenta: “Verificar últimas queries lentas no banco de dados”.

Engenheiro vê que a query de usuários está executando 100x mais vezes que o normal.

Engenheiro identifica a causa raiz: query N+1 no código novo.

Tempo total: 8 minutos.

Desafios e Considerações:

Fornecer contexto pré-compilado exige investimento significativo em automação e integração de ferramentas. Adicionalmente, se o contexto for incorreto ou enganoso, pode levar o engenheiro na direção errada. A solução é começar com contexto simples e bem validado. Ex.: dados de deployment e evoluir gradualmente para contexto mais complexo. Ex.: análise de causa raiz com IA.

Reduzir o Tempo de Remediação Através de Automação de Runbooks

O tempo de remediação é o tempo gasto implementando a solução. Para problemas conhecidos, isso pode ser automatizado completamente. Para problemas novos, pode ser acelerado através de runbooks bem documentados.

Metodologia de Implementação:

A automação de runbooks funciona criando scripts ou workflows que implementam a solução automaticamente:

Runbooks Parametrizados: um runbook não é apenas um documento; é um workflow executável. Por exemplo, em vez de um runbook que diz “Reinicie o serviço X”, é um workflow que automaticamente reinicia o serviço, aguarda que ele fique saudável, e verifica se o problema foi resolvido.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Aprovação Humana Condicional: para ações de alto risco, exija aprovação humana antes de executar. Para ações de baixo risco, execute automaticamente e notifique o engenheiro depois.

Rollback Automático: se a automação causar mais dano do que bem, implemente rollback automático. Por exemplo, se um restart de serviço não resolver o problema em 2 minutos, revert para a versão anterior.

Exemplo Prático - Remediação Automatizada:

Considere um incidente onde a memória de um serviço está alta. O runbook automatizado:

- Verifica se há vazamento de memória, comparando memória ao longo do tempo.
- Se não há vazamento, reinicia o serviço. Ação de baixo risco é executada automaticamente.
- Aguarda 30 segundos e verifica se o serviço está saudável.
- Se saudável, resolve o incidente automaticamente.
- Se não saudável, escala para um engenheiro com contexto completo.

Tempo de remediação: 35 segundos vs. 15+ minutos se um engenheiro tivesse que fazer manualmente.

Desafios e Considerações:

A automação de runbooks exige confiança. Se a automação falhar e causar mais problemas, a confiança é perdida e a equipe voltará a fazer tudo manualmente. A solução é começar com ações de baixo risco e evoluir gradualmente.

Reduzir o Tempo de Verificação Através de Testes Automatizados

O tempo de verificação é o tempo gasto confirmando que a solução funcionou. Isso pode ser automatizado através de testes que verificam se o serviço retornou ao normal.

Metodologia de Implementação:

A verificação automatizada funciona definindo critérios claros de “resolvido” e verificando automaticamente se esses critérios foram atendidos:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Critérios de Resolução: para cada tipo de incidente, defina claramente o que significa “resolvido”. Exemplos: “Taxa de erro < 0.1%”, “Latência P99 < 200ms”, “Taxa de sucesso de checkout > 99%”.

Testes Automatizados: implemente testes que verificam esses critérios. Por exemplo, um teste que tenta fazer uma compra de teste e verifica se foi bem-sucedida.

Resolução Automática: se os critérios de resolução forem atendidos, o incidente é automaticamente resolvido. Se não forem, o incidente permanece aberto e é escalado novamente.

Exemplo Prático - Verificação Automatizada:

Após a remediação de um incidente de checkout, o sistema automaticamente:

- Executa um teste de compra de teste.
- Verifica se a taxa de erro de checkout está < 0.1%.
- Verifica se a latência do checkout está < 500ms.
- Se todos os critérios forem atendidos, marca o incidente como resolvido.
- Se algum critério não for atendido, mantém o incidente aberto.

Isso elimina o tempo de verificação manual.

PagerDuty: Plataforma para Reduzir MTTR

O PagerDuty é uma plataforma de gerenciamento de incidentes projetada especificamente para reduzir o MTTR através de automação, orquestração e inteligência. Enquanto ferramentas como Prometheus e Grafana focam em monitoramento e observabilidade, o PagerDuty foca no que acontece depois que um alerta é gerado.

Funcionalidades Principais do PagerDuty para Reduzir MTTR

Escalação Inteligente e Notificações Multicanal

O PagerDuty permite definir políticas de escalação que garantem que o alerta chegue à pessoa certa no momento certo:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Políticas de Escalação: defina quem deve ser notificado primeiro, e se não responder em X minutos, quem deve ser notificado em seguida. Por exemplo: notificar o engenheiro on-call de frontend; se não responder em 5 minutos, notificar o lead de frontend; se ainda não responder em 5 minutos, notificar o CTO.

Notificações Multicanal: não confie em um único canal. O PagerDuty pode notificar via Slack, SMS, e-mail, push notification e chamada telefônica, garantindo que o engenheiro seja alcançado.

Escalação Condicional: a escalação pode ser condicional baseada na severidade do incidente, na hora do dia ou em outras variáveis. Por exemplo, incidentes críticos durante o horário comercial vão para o engenheiro on-call; incidentes críticos à noite vão para o engenheiro sênior.

Agrupamento e Correlação de Alertas

O agrupamento de alertas é essencial para reduzir o ruído. O PagerDuty oferece:

Agrupamento Automático: múltiplos alertas relacionados são automaticamente agrupados em um único incidente, reduzindo a fragmentação.

Correlação de Eventos: o PagerDuty correlaciona eventos de múltiplas fontes para identificar incidentes relacionados. Por exemplo, se um alerta de “Banco de Dados Inacessível” é disparado, todos os alertas de “Taxa de Erro Alta” para serviços dependentes são correlacionados e agrupados.

Supressão Inteligente: alertas de baixa prioridade são suprimidos quando um alerta de alta prioridade já está ativo, reduzindo o ruído.

Automação de Resposta a Incidentes

O PagerDuty integra-se com ferramentas de automação para executar ações automaticamente:

Webhooks e Integrações: quando um incidente é criado, o PagerDuty pode disparar webhooks que acionam automações em ferramentas como Ansible, Terraform, ou scripts customizados.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Runbooks Executáveis: o PagerDuty permite anexar runbooks a incidentes. Esses runbooks podem ser executados automaticamente ou manualmente pelo engenheiro.

Automação Condicional: a automação pode ser condicional. Por exemplo: se o incidente é de tipo “Memória Alta”, tente reiniciar o serviço automaticamente; se é de tipo “Taxa de Erro Alta”, escale para um engenheiro.

Contexto Enriquecido e Investigação Acelerada

O PagerDuty fornece contexto detalhado quando um incidente é criado:

Timeline de Eventos: o PagerDuty mostra uma timeline completa de eventos relacionados ao incidente, incluindo quando o alerta foi gerado, quando foi reconhecido, quando foi escalado, etc.

Dados de Contexto: o PagerDuty pode ser integrado com ferramentas de observabilidade para anexar dados relevantes ao incidente. Ex: gráficos de Grafana, traces de Datadog, logs relevantes.

Histórico de Incidentes: o PagerDuty mantém um histórico completo de todos os incidentes anteriores, permitindo que engenheiros(as) vejam rapidamente se um incidente similar ocorreu antes e como foi resolvido.

Análise Pós-Incidente e Aprendizado Contínuo

O PagerDuty facilita postmortems e análise pós-incidente:

Relatórios de Incidentes: o PagerDuty gera relatórios automáticos que mostram o MTTR, o MTTD (Mean Time To Detect), o MTTR por tipo de incidente, etc.

Postmortems Estruturados: o PagerDuty fornece templates para postmortems, garantindo que a análise seja consistente e completa.

Rastreamento de Ações: o PagerDuty rastreia ações de acompanhamento (ex: “Melhorar o alerta de X”) e garante que sejam concluídas.

Exemplo Prático: Reduzindo MTTR com PagerDuty

Considere um cenário onde um serviço crítico de API fica indisponível:

Sem PagerDuty (MTTR = 45 minutos):

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- Prometheus gera um alerta (T=0).
- Alerta é enviado para Slack (T=1).
- Engenheiro on-call vê a mensagem de Slack 10 minutos depois (T=10).
- Engenheiro reconhece o alerta manualmente (T=12).
- Engenheiro abre Grafana, Datadog, e logs para investigar (T=15).
- Engenheiro identifica que a causa é um deployment recente (T=25).
- Engenheiro faz rollback do deployment (T=30).
- Engenheiro verifica que o serviço está recuperado (T=45).

Com PagerDuty (MTTR = 12 minutos):

- Prometheus gera um alerta (T=0).
- PagerDuty recebe o alerta e o correlaciona com alertas anteriores (T=1).
- PagerDuty dispara uma automação: tenta reiniciar o serviço (T=2).
- Reinicio falha, então PagerDuty escala para o engenheiro on-call via Slack + SMS (T=3).
- Engenheiro reconhece o alerta no PagerDuty (T=4).
- PagerDuty apresenta contexto enriquecido: “Incidente similar em 10/11/2025 foi causado por deployment Y. Rollback resolveu em 5 minutos.” (T=5).
- Engenheiro executa o runbook de rollback automaticamente (T=6).
- PagerDuty verifica automaticamente que o serviço está recuperado (T=12).

A diferença é dramática: 45 minutos vs. 12 minutos. Em uma plataforma processando US$ 10 mil por minuto, isso economiza US$ 330 mil.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

MERCADO, CASES E TENDÊNCIAS

O mercado de gerenciamento de incidentes está evoluindo rapidamente. Enquanto o PagerDuty foi pioneiro neste espaço, novos concorrentes como Opsgenie adquirido pela Atlassian Incident.io e OnCall (da Grafana) estão oferecendo alternativas. No entanto, o PagerDuty continua sendo o líder de mercado, com mais de 15 mil clientes incluindo Netflix, Slack, Airbnb e GitHub.

Um case notável é o da Slack. Ao implementar o PagerDuty e otimizar suas políticas de escalação e automação, a Slack reduziu seu MTTR de 45 minutos para 12 minutos para incidentes críticos. Isso não apenas melhorou a confiabilidade do serviço, mas também reduziu o burnout da equipe de on-call, pois os engenheiros gastam menos tempo em investigação manual.

As tendências para 2025 e além incluem:

AIOps Integrado: as plataformas de gerenciamento de incidentes estão integrando cada vez mais IA para análise automática de causa raiz, sugestão de remediações e previsão de incidentes.

Automação End-to-End: a tendência é automatizar não apenas a detecção e a resposta inicial, mas todo o ciclo de vida do incidente, incluindo postmortems e rastreamento de ações.

Observabilidade Integrada: as plataformas estão integrando observabilidade (logs, métricas, traces) diretamente no gerenciamento de incidentes, eliminando a necessidade de alternar entre ferramentas.

Para se aprofundar, explore os seguintes recursos:

Documentação: PagerDuty Documentation.

Artigo: PAGERDUTY. MTTR: What It Is and Why It Matters.

Case Study: Transforming critical work at over 30,000 organizations

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos um dos conceitos mais críticos para operações de TI modernas: o MTTR. Começamos entendendo o que é MTTR e por que é importante, não apenas como uma métrica, mas como um indicador direto do impacto financeiro e da maturidade operacional.

Aprofundamos na metodologia de medição do MTTR, decompondo-o em seus cinco componentes principais: detecção, resposta, diagnóstico, remediação e verificação, apresentando uma abordagem rigorosa para mensurá-lo consistentemente. Você aprendeu sobre os desafios comuns na medição e como evitá-los.

Em seguida, apresentamos cinco estratégias avançadas para reduzir o MTTR, cada uma com metodologia clara de implementação e exemplos práticos.

Finalmente, exploramos como o PagerDuty, uma plataforma de gerenciamento de incidentes líder de mercado, implementa essas estratégias através de funcionalidades como escalação inteligente, agrupamento de alertas, automação de resposta e análise pós-incidente.

Com este conhecimento, você está preparado(a) para implementar um programa de redução de MTTR em sua organização, resultando em serviços mais confiáveis, equipes menos estressadas e impacto financeiro significativo.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

INCIDENT.IO. **The practical guide to incident management**. Disponível em: https://incident.io/guide. Acesso em: 02 fev. 2026.

MANDIANT. **Best practices for incident response planning**. Mandiant Best Practices. Disponível em: https://services.google.com/fh/files/misc/mandiant_incident_response_best_practices_2025.pdf. Acesso em: 02 fev. 2026.

PAGERDUTY. **Customers.** Disponível em: https://www.pagerduty.com/customers/. Acesso em: 02 fev. 2026.

PAGERDUTY. **MTTR: What It Is and Why It Matters**. Disponível em: https://www.pagerduty.com/resources/devops/learn/what-is-mttr/. Acesso em: 02 fev. 2026.

PAGERDUTY. **PagerDuty Documentation.** Disponível em: https://support.pagerduty.com/. Acesso em: 02 fev. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE

MTTR. Mean Time To Resolution. PagerDuty. Gerenciamento de Incidentes. Automação de Resposta. Escalação Inteligente. Observabilidade.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com